IDU decode cheat sheet

HIPV core, D stage. Sources: `rtl/core/idu/{idu,idudec,extend}.sv`, `config/hipv_pkg.sv`.

1. Instruction fields

31..25	24..20	19..15	14..12	11..7	6..0
funct7	rs2	rs1	funct3	rd	opcode

Signal	Bits	Use
`OpD`	`InstrD[6:0]`	main decoder select
`RdD`	`InstrD[11:7]`	destination register
`Funct3D`	`InstrD[14:12]`	ALU decoder; passed on (branch type, load/store size)
`Rs1D`	`InstrD[19:15]`	regfile A1
`Rs2D`	`InstrD[24:20]`	regfile A2
`Funct7b5D`	`InstrD[30]`	ADD/SUB and SRL/SRA (only funct7 bit used)
–	`InstrD[31:7]`	into `extend`

2. Main decoder: opcode → control word

Control word (15 bits, MSB first):
`{RegWrite, ImmSrc[2:0], ALUSrcA[1:0], ALUSrcB, MemWrite, ResultSrc[1:0], Branch, Jump, Jalr, ALUOp[1:0]}`

Instr	Opcode	RegW	ImmSrc	SrcA	SrcB	MemW	ResSrc	Br	Jmp	Jalr	ALUOp
R-type	0110011	1	I	RF	RD2	0	ALU	0	0	0	10
I-ALU	0010011	1	I	RF	imm	0	ALU	0	0	0	10
load	0000011	1	I	RF	imm	0	MEM	0	0	0	00
store	0100011	0	S	RF	imm	1	ALU	0	0	0	00
branch	1100011	0	B	RF	RD2	0	ALU	1	0	0	01
jal	1101111	1	J	RF	RD2	0	PC4	0	1	0	00
jalr	1100111	1	I	RF	imm	0	PC4	0	0	1	00
lui	0110111	1	U	ZERO	imm	0	ALU	0	0	0	00
auipc	0010111	1	U	PC	imm	0	ALU	0	0	0	00
other	–	all zero → nop (fence, ecall, csr, …; no illegal-instruction trap)									

- `jal` and branch targets use the dedicated PC + imm adder in Execute; `jalr` target = `rs1 + imm` on the ALU.
- `lui` = `0 + imm`; `auipc` = `PC + imm`.
- Control outputs are only meaningful when `ValidD` = 1; later stages must qualify side effects with it.

3. Select-signal encodings (`hipv_pkg`)

Signal	Values
`ImmSrcD`	`000` I, `001` S, `010` B, `011` J, `100` U
`ALUSrcAD`	`00` RF `RD1`, `01` PC, `10` zero
`ALUSrcBD`	`0` `RD2`, `1` `ImmExtD`
`ResultSrcD`	`00` ALU, `01` MEM, `10` PC+4
`ForwardAE/BE` (hazard unit)	`00` D/E reg, `01` `ResultW`, `10` `ForwardDataM`

4. ALU decoder: `ALUControlD`

If `ALUOp` ≠ `10`: `ALU_ADD` (address/target computation). If `ALUOp` = `10` (R-type, I-ALU):

funct3	Operation → ALUControlD	Condition
000	add 0000 / sub 0001	SUB only if InstrD[30] & OpD[5]
001	sll 0010	
010	slt 0011	
011	sltu 0100	
100	xor 0101	
101	srl 0110 / sra 0111	SRA if InstrD[30] (sra and srai)
110	or 1000	
111	and 1001	

Why OpD[5] for sub: in addi with a negative immediate, InstrD[30] is an immediate bit, not funct7. OpD[5] is 1 for R-type (0110011) and 0 for I-ALU (0010011).

5. Immediate extension (extend)

All immediates sign-extend from InstrD[31]; i = InstrD. B and J have implicit bit 0 = 0.

Fmt	ImmExtD	Used by
I	{{20{i[31]}}, i[31:20]}	I-ALU, load, jalr
S	{{20{i[31]}}, i[31:25], i[11:7]}	store
B	{{20{i[31]}}, i[7], i[30:25], i[11:8], 1'b0}	branch
J	{{12{i[31]}}, i[19:12], i[20], i[30:21], 1'b0}	jal
U	{i[31:12], 12'b0}	lui, auipc
other	32'b0	–

6. Stage handshake and flush

```
ReadyD = ~ValidDReg | ReadyE   (accept when empty or Execute drains)
EnFD = ValidF & ReadyD         (F/D data registers load)
ValidD = ValidDReg & ~PCSrcE   (taken branch/jump in E squashes D)
ValidDReg <= ValidF when ReadyD   (reset → 0)
```

- Backpressure (ReadyE = 0) holds the instruction in D and stalls ReadyD.
- PCSrcE gates ValidD combinationally; the bubble is committed at the next edge via ValidF (gated by PCSrcE in ifuctrl).

7. Register file

32×32, two combinational read ports (Rs1D, Rs2D), one write port from W (RegWriteW, RdW, ResultW). x0 reads as zero and ignores writes. Write-first: a same-cycle write from W is bypassed to RD1D/RD2D, so no extra WB→D forwarding is needed. The array is not reset.

8. Worked examples

1. addi x5, x1, -4 = 0xFFC08293

Field	Value	Decode
opcode [6:0]	0010011	I-ALU: RegWrite=1, ImmSrc=I, SrcA=RF, SrcB=imm, ResultSrc=ALU, ALUOp=10
rd [11:7]	5	RdD
funct3 [14:12]	000	add/sub branch
rs1 [19:15]	1	Rs1D
imm [31:20]	0xFFC	ImmExtD = 0xFFFFFFFC
InstrD[30]	1	ignored, since OpD[5] = 0 → ALUControlD = ADD

2. `sw x6, 8(x2)` = `0x00612423`

Field	Value	Decode
opcode `[6:0]`	`0100011`	store: RegWrite=0, ImmSrc=S, SrcB=imm, MemWrite=1, ALUOp=00
imm[4:0] `[11:7]`	`01000`	low part of offset (not `RdD`; unused since RegWrite=0)
funct3 `[14:12]`	`010`	word access (passed on as `Funct3D`)
rs1 `[19:15]`	2	`Rs1D` = base address
rs2 `[24:20]`	6	`Rs2D` = store data
imm[11:5] `[31:25]`	`0000000`	high part of offset
`ImmExtD`	`0x00000008`	S format
`ALUControlD`	`ALU_ADD`	`ALUOp` $\neq$ `10`: address = `rs1 + imm`